Extending CT-Prover: Detecting Time-Variant Instruction Side-Channel Sources

Zach Riback & Sam Millman

Introduction

- Problem:
  - Cryptographic operations are required to be run in constant time in order to prevent side channels from leaking information.
  - It is possible to extract sensitive information using attacks that require specific timing in order to successfully exploit.
    - Cryptographic keys, secret messages, etc.
- Solution:
  - Prove cryptographic operations are not being run in a non-constant time.
  - CT-Prover: Taint analysis tool that aims to do this, with some limitations. We aim to improve functionality to include other possible sources of non-constant time functions.

Background

- The importance of constant time:
  - Constant time, commonly written as O(1), means that a function's time to run does not change with different variables.
    - Example: in a function "foo(x)", foo will take the same amount of time to run, no matter what the value of the variable "x" might be.
  - In the world of cryptography, certain variables hold secret information that can fundamentally break encryption and decryption standards.
  - If a function's runtime duration is different over the course of multiple iterations, an adversary can feed different inputs into the function to extract this secret information via a side channel.

Background

- What is a side channel?
  - Unintended pathway in which information is leaked from a system.
  - Typically, it is caused by an indirect output, separate from the intentional design of some software, hardware, etc.
    - Example: The CPU draws more power when adding then subtracting. Therefore, an adversary can learn whether a program is doing an addition rather than a subtraction based on input.
    - There are multiple kinds of side channels, from timing based, to cache based, to power based, and more. They usually stem from shared resources.
  - For our purposes, we are looking to detect timing differences between iterations of a function, based on varying inputs.
- Side channels have been thoroughly explored.
  - “Remote timing attacks are still practical”

Background

- Division instructions:
  - In x86 assembly, a DIV instruction is used to do division.
    - This is NOT a constant time instruction, and therefore should not be used for timing sensitive functions.
  - The DIV instruction takes a varying number of clock cycles to complete. This variance is what causes timing differences.
  - Example: Related work, Divide and Surrender, observes that a modulo instruction in a loop is compiled using a DIV instruction.

Background

- Safety Verification
  - The process of proving that a system or program never reaches a “bad” state.
  - This includes things like safe coding practices in memory, types, concurrency, and other secure coding practices.
- LLVM
  - Full compiler framework. Uses an Intermediate Representation (IR) which is a low-level, typed, assembly adjacent language that is platform independent.
  - Key part of universal code analysis that can translate code into machine code and detect bad coding practices in machine code.
- Clang
  - Compiler front end for C family of languages. Part of LLVM.
  - Handles syntax checking and semantic analysis. Also provides libraries for static analyzers and other tools that are very robust.
    - Important in the role of code optimization.

Background

- Taint Analysis - Analysis of how untrusted or sensitive data flows through a program (hence, tainted data).
  - Examples of tainted data: User input, network data, cryptographic keys.
  - Analyzes how the data moves through variables, functions, and memory.
- Static Taint Analysis
  - Examines code without running it, which can cover all possible paths but can be possibly unreliable, producing false positives.
  - Used in compilers and other security tools.
- Dynamic Taint Analysis
  - Tracks data flow while the program is running.
  - Great for precise runs of the program, but limited in which paths are actually executed.

Related Work

- Divide and Surrender
  - A side channel is explored that exists due to a compiler issue that emitted “DIV” instructions instead of constant-time Barrett reductions.
    - Vulnerability in Simultaneous Multithreading (SMT)
  - In a specific implementation of HQC, or Hamming Quasi-Cyclic, if the divisor is known at compile time, compilers may replace division with a Barrett reduction, which is a known constant time function.
    - However, if done inside of a loop or some other specific contexts, it is replaced with a “DIV” instruction. This creates the side channel.
- This is an example of a real world vulnerability caused by a “DIV” instruction used after compilation.

Related Work

- KyberSlash
  - Kyber - A NIST-standardized post-quantum encapsulation mechanism.
  - In the targeted implementation of Kyber, division instructions were used.
    - NOT constant time!
  - Coefficients of polynomials that were instrumental in the encapsulation mechanism, were used as the divisor, resulting in leaking of these coefficients.
  - Larger values resulted in a longer execution time, creating a timing side-channel.

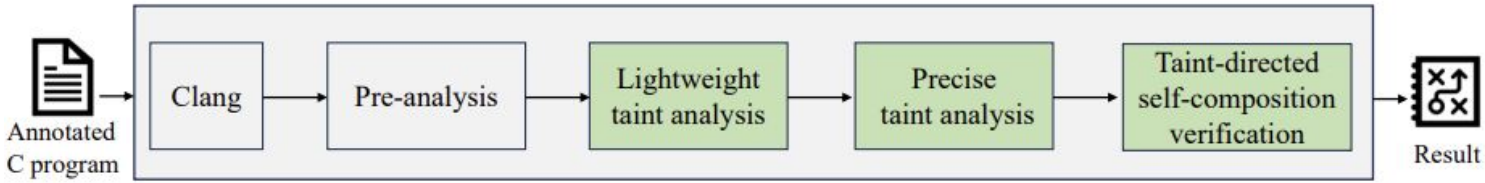
- Practical example of a real-world scenario where a division instruction was used in a cryptography function, resulting in leakage of a secret value.

Related Work

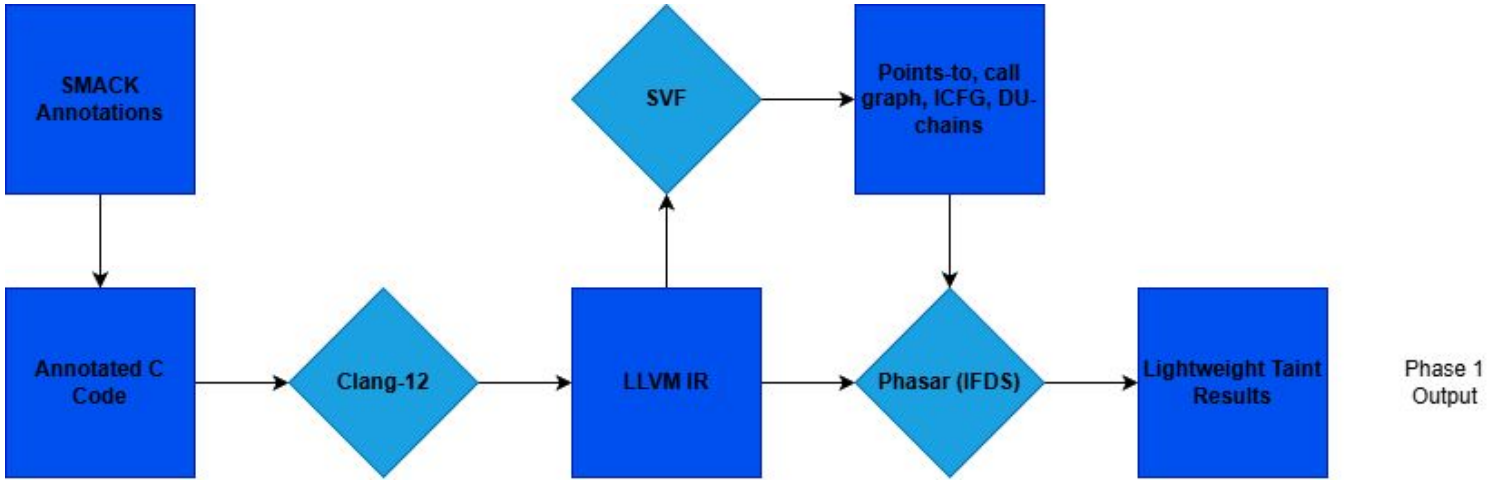
- CT-Verif
  - Tool that is used to automatically verify whether cryptographic implementations are constant-time.
  - Translates LLVM Intermediate Response into Boogie intermediate verification language.
    - Then, verification is handled by solvers, such as Corral and Z3, which verify that there are not branches of code dependent on secret control-flow or memory access exits.

Baseline - CT_Prover

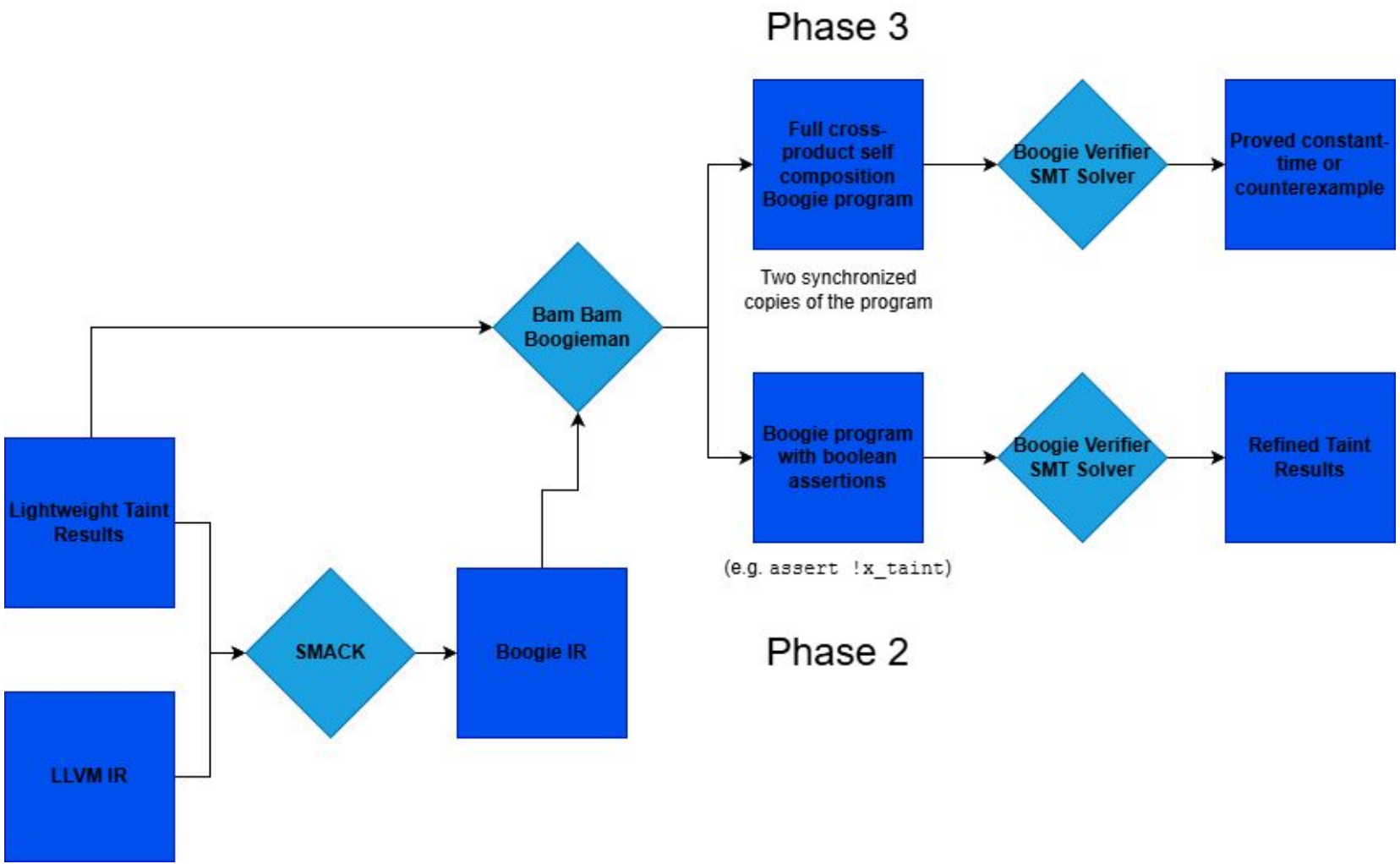
- Based on ct-verif



```c
void function_wrapper(int secret, int public) {
    vfct_taintseed(secret);
    public_in(__SMACK_value(public));

    (void)bad_function(secret, public);
}
```




```
br i1 %5, label %6, label %9, !dbg !30, !psr.id !31, !Tainted !32
br i1 %16, label %17, label %22, !dbg !61, !psr.id !62, !Tainted !32
%27 = load i8, i8* %26, align 1, !dbg !87, !psr.id !90, !Tainted !32
%37 = sdiv i32 %0, 10, !dbg !103, !psr.id !104, !Tainted !32
%39 = srem i32 %0, 10, !dbg !111, !psr.id !112, !Tainted !32
```


Phase 3
Full cross-product self composition Boogie program
Boogie Verifier SMT Solver
Proved constant-time or counterexample
Two synchronized copies of the program
Bam Bam Boogieman
Boogie program with boolean assertions
Boogie Verifier SMT Solver
Refined Taint Results
(e.g. assert !x_taint)
Lightweight Taint Results
SMACK
Boogie IR
LLVM IR
Phase 2

Contributions

- Detect **tainted divisions** as well as branches, data accesses, and loops
  - “Note that our methodology is generic and could be adapted to handle [time variant instructions (e.g. integer divisions)] by listing all the timing-sensitive operations”

- Make the tool easier to use
  - Update documentation
  - Fix various code issues
  - Implement capability to show source lines where tainted operations are happening

Fixes and Improvements

```
PHASAR  ="/home/luwei/phasar/build/tools/phasar-llvm/phasar-llvm"
BAMPATH = "/home/luwei/bam-"
```

```
import os

CTPROVER_ROOT = os.getenv('PROJECT_ROOT')

PHASAR  = CTPROVER_ROOT + "/phasar/build/tools/phasar-llvm/phasar-llvm"
BAMPATH = CTPROVER_ROOT + "/bam/bam-"
```

- Added READMEs
- Actually document all dependencies
- Improved scripts
  - `vfct_all.py`
  - `vfct_only1.py`
  - `vfct_123.py`
  - `vfct_1_all.py`
  - `vfct_123_all.py`
  - `summarize_results.py`
  - `annotate.py`

```
auto getdumpresult(std::set<llvm::Instruction*>& res, std::set<llvm::Instruction*>& res2, std::set<

    std::set<llvm::Module *> mds = IRDB.getAllModules();
    for(auto M : mds){
        for(auto &F : *M)
            for(auto &BB : F)
                for(auto &I : BB){
                    if(llvm::isa<llvm::DbgInfoIntrinsic>(&I))
                        continue;

                    std::vector<const llvm::Value *> TaintSeeds;
                    if(const auto *Load = llvm::dyn_cast<llvm::LoadInst>(&I)){
                        TaintSeeds.push_back(Load->getPointerOperand());
                    } else if(const auto *Store = llvm::dyn_cast<llvm::StoreInst>(&I)){
                        TaintSeeds.push_back(Store->getPointerOperand());
                    }else if(const llvm::BranchInst *Br = llvm::dyn_cast<llvm::BranchInst>(&I)){
                        if (Br->isUnconditional())
                            continue;
                        TaintSeeds.push_back(Br->getCondition());
                    }else if(const auto *BinOp = llvm::dyn_cast<llvm::BinaryOperator>(&I)){
                        switch (BinOp->getOpcode()) {
                        case llvm::Instruction::SDiv:
                        case llvm::Instruction::UDiv:
                        case llvm::Instruction::FDiv:
                        case llvm::Instruction::SRem:
                        case llvm::Instruction::URem:
                        case llvm::Instruction::FRem:
                            TaintSeeds.push_back(BinOp->getOperand(0));
                            TaintSeeds.push_back(BinOp->getOperand(1));
                            break;
                        default:
                            break;
                        }
```

- Update Phasar to treat division/remainder instructions as taint seeds
  - Always check their operands to see if they are tainted
  - `sdiv, udiv, fdiv, srem, urem, frem`

Demo

Results and Findings

Found **3** locations with **8** tainted divisions in CT_Prover's included benchmarks

- BearSSL_RSA_i32_decrypt (1) (previously not CT)
- MAC-then-Encode-then-CBC-Encrypt (4) (previously CT)
- OpenSSL_ssl3_cbc_digest_record (3) (previously CT)

87 total benchmarks

- 72 constant time…70 can be determined with only the first phase
- 15 not constant time

```
br_i32_decode(uint32_t *x, const void *src, size_t len)
{
    const unsigned char *buf;
    size_t u, v;

    buf = src;
    u = len;
    v = 1;
    for (;;) {
        if (u < 4) {    // TAINTED BRANCH
            uint32_t w;

            if (u < 2) {    // TAINTED BRANCH
                if (u == 0) {    // TAINTED BRANCH
                    break;
                } else {
                    w = buf[0];
                }
            } else {
                if (u == 2) {    // TAINTED BRANCH
                    w = br_dec16be(buf);
                } else {
                    w = ((uint32_t)buf[0] << 16)
                        | br_dec16be(buf + 1);
                }
            }
            x[v ++] = w;
            break;
        } else {
            u -= 4;
            x[v ++] = br_dec32be(buf + u);
        }
    }
    x[0] = (len / 4) << 5 + 32;    // TAINTED DIVISION
}
```

- BearSSL library
- `size_t len` is derived from a secret key
  - it is the length of *p* or *q* factors of the RSA private key in bytes
- In the call graph of

```
br_rsa_i32_private(unsigned char *x,
                   const br_rsa_private_key *sk
                  )
```

```
int crypto_auth_ct(unsigned char *out,const unsigned char *in,unsigned long publen,
  int pub_blocks, index_a, index_b, index_c, i, j;
  unsigned long long bits;
  unsigned char mac_computed[CRYPTO_BYTES];
  unsigned char hash_state[CRYPTO_BYTES];
  unsigned char block_h[BLOCKBYTES];

  pub_blocks = (publen - CRYPTO_BYTES - 2) / BLOCKBYTES;
  index_a    = (inlen - CRYPTO_BYTES)   / BLOCKBYTES;       //  TAINTED DIVISION
  index_c    = (inlen - CRYPTO_BYTES)   % BLOCKBYTES;       //  TAINTED DIVISION
  index_b    = index_a + (1 & constant_time_ge(index_c,56));

  // Initialize the hash state
  for (i = 0;i < CRYPTO_BYTES;i++) {
    hash_state[i] = hmac_iv[i];
  }
  ...
}
```

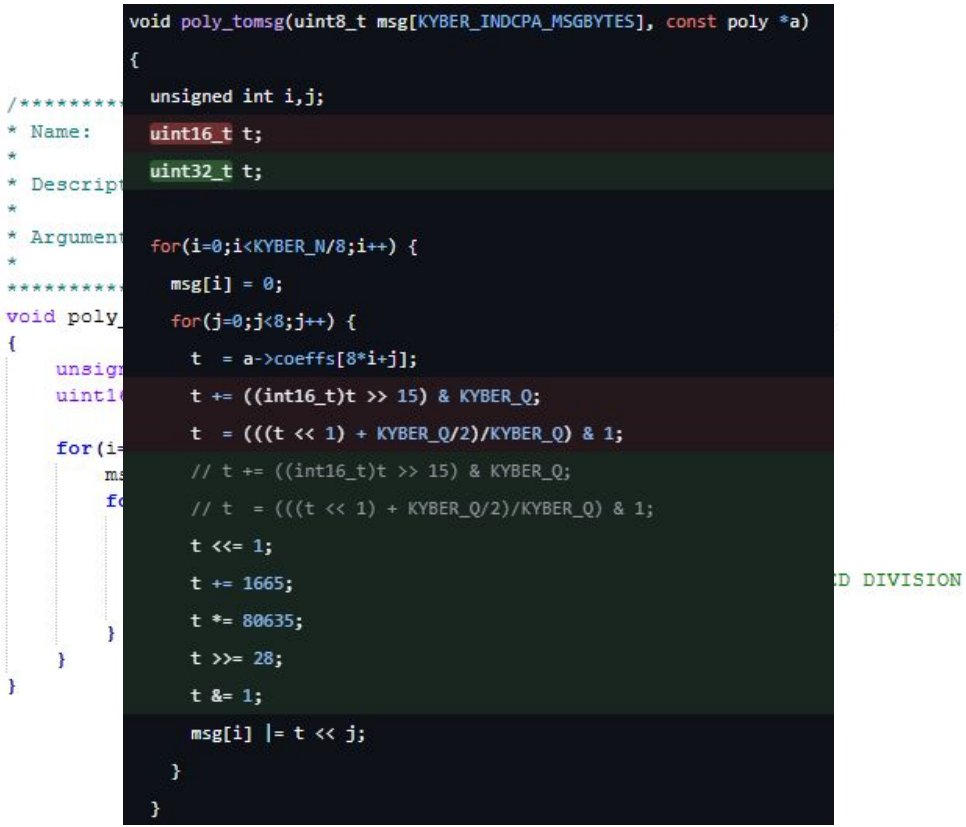
```
int crypto_auth_verify(const unsigned char *in,unsigned long publen, unsigned long inlen,const unsigned char *k
{
  unsigned char correct[32];
  unsigned char mac[32];

  // The idea here is to scan through all possible locations for the
  // MAC, pretending to copy out CRYPTO_BYTES bytes starting from each
  // of them.
  int i, j;
  unsigned int mac_end = inlen; // secret
  unsigned int mac_start = mac_end - CRYPTO_BYTES; // secret
  unsigned scan_start = 0;
  unsigned char rotate_offset; // secret

  // We first refine the starting point of the scan
  if (publen > CRYPTO_BYTES + 255 + 1) {
    scan_start = publen - (CRYPTO_BYTES + 255 + 1);
  }

  // And compute the offset around which we will need to rotate
  rotate_offset = (mac_start - scan_start) % CRYPTO_BYTES;    // TAINTED DIVISION

  ...

  for (i = 0;i < CRYPTO_BYTES;i++) {
    unsigned char offset = (CRYPTO_BYTES - rotate_offset + i) % CRYPTO_BYTES; // secret    // TAINTED DIVISION
    for (j = 0;j < CRYPTO_BYTES;j++) {
      mac[j] |= correct[i] & constant_time_eq_8(j,offset);
    }
```

- MAC-then-Encode-then-CBC-Encrypt
- Decrypts a MAC and verifies it with the message
- `inlen` is the length of the plaintext

```
void ssl3_cbc_digest_record(const EVP_MD_CTX *ctx,
                            unsigned char *md_out,
                            size_t *md_out_size,
                            const unsigned char[header13],
                            const unsigned char *data,
                            size_t data_plus_mac_size,
                            size_t data_plus_mac_plus_padding_size,
                            const unsigned char *mac_secret,
                            unsigned mac_secret_length, char is_sslv3)
{
    ...
    mac_end_offset = data_plus_mac_size + header_length - md_size;

    c = mac_end_offset % md_block_size;     // TAINTED DIVISION

    index_a = mac_end_offset / md_block_size;      // TAINTED DIVISION

    index_b = (mac_end_offset + md_length_size) / md_block_size;        // TAINTED DIVISION

    bits = 8 * mac_end_offset;
    if (!is_sslv3) {
        bits += 8 * md_block_size;
        memset(hmac_pad, 0, md_block);
        ...
    }
}
```

- OpenSSL
- Computes the MAC after decrypting a SSLv3 CBC-mode record
- `mac_end_offset` is tainted
  - depends on the plaintext length after padding is removed
- If learned, known to enable certain timing attacks (Lucky13)

Bonus Benchmark



```
void poly_tomsg(uint8_t msg[KYBER_INDCPA_MSGBYTES], const poly *a)
{
  unsigned int i,j;
  uint16_t t;
  uint32_t t;

  for(i=0;i<KYBER_N/8;i++) {
    msg[i] = 0;
    for(j=0;j<8;j++) {
      t  = a->coeffs[8*i+j];
      t += ((int16_t)t >> 15) & KYBER_Q;
      t  = (((t << 1) + KYBER_Q/2)/KYBER_Q) & 1;
      // t += ((int16_t)t >> 15) & KYBER_Q;
      // t  = (((t << 1) + KYBER_Q/2)/KYBER_Q) & 1;
      t <<= 1;
      t += 1665;
      t *= 80635;
      t >>= 28;
      t &= 1;
      msg[i] |= t << j;
    }
  }
}
```

- KyberSlash Vulnerability
- `poly_tomsg` converts polynomial coefficients
  - the coefficients are secret

- The exploitability of these tainted divisions has been demonstrated!

Future Work

- Continue to improve the tool
  - documentation, scripting, etc.
- More side channel sources
  - Floating point operations?
- Further analysis
  - Investigate our identified division side-channel sources
  - Phase 2 and 3

Conclusion

- Searching for time-based side channel vulnerabilities is important
  - including those from divisions (KyberSlash, Divide and Surrender)
- CT_Prover ended up being quite difficult to work with
  - fixed code issues, improved (added) documentation
- Successfully add capabilities
  - detect tainted divisions
  - mark tainted lines
- Showed the presence of tainted divisions in CT_Prover’s benchmarks

Works Cited

Almeida, J. B., Barbosa, M., Barthe, G., Dupressoir, F., & Emmi, M. (2016). Verifying {Constant-Time} Implementations. In *25th USENIX Security Symposium (USENIX Security 16)* (pp. 53-70).

Bernstein, D. J., Bhargavan, K., Bhasin, S., Chattopadhyay, A., Chia, T. K., Kannwischer, M. J., ... & Tamvada, G. (2024). KyberSlash: Exploiting secret-dependent division timings in Kyber implementations. *Cryptology ePrint Archive*.

Brumley, B. B., & Tuveri, N. (2011, September). Remote timing attacks are still practical. In *European Symposium on Research in Computer Security* (pp. 355-371). Berlin, Heidelberg: Springer Berlin Heidelberg.

Cai, L., Song, F., & Chen, T. (2024). Towards efficient verification of constant-time cryptographic implementations. *Proceedings of the ACM on Software Engineering*, *1*(FSE), 1019-1042.

Schröder, R. L., Gast, S., & Guo, Q. (2024). Divide and Surrender: Exploiting Variable Division Instruction Timing in {HQC} Key Recovery Attacks. In *33rd USENIX Security Symposium (USENIX Security 24)* (pp. 6669-6686).

Questions?